Reviewer Pack — Litvinov-Maslov Entropy Bound 3β·log(n) for Tropical Doublin…

Entry 04c00d1d0cca · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 10:07:20 UTC

Litvinov-Maslov Entropy Bound 3β·log(n) for Tropical Doubling Defect

Entry ID: `04c00d1d0cca` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 10:07:20 UTC

1. Conjecture

Field A (mathematical branch): Maslov dequantization **Field B** (complexity object): Tropical Fourier analysis

Statement:

> Let f:Z_n→R be any tropical polynomial with bounded coefficients, let g(x):=β·log(Σ_{y∈Z_n} exp((f(y)+f(x−y))/β)) be the β-softmax self-convolution, and define TFT_β(h)[k]:=β·log(|Σ_{x∈Z_n} exp(h(x)/β)·exp(−2πikx/n)|), MFC_β(h):=min_{k≠0} TFT_β(h)[k]. Then for every f and every n≥2 at fixed β=5, the doubling defect Δ(f,β,n):=|MFC_β(g)−2·MFC_β(f)| satisfies Δ(f,β,n) ≤ 3·β·log(n), where the constant 3 = 1 (one n-term softmax in forming g) + 2 (one n-term softmax inside each of the two MFC_β(f) terms) is the closed-form Litvinov-Maslov entropy correction. A single (n,seed) instance with Δ > 3·β·log(n) refutes the conjecture.

Rationale (proposer's reasoning):

> Litvinov-Maslov says β·log(Σ_{i=1}^n e^{a_i/β}) lies in [max a_i, max a_i + β·log(n)], so every n-term softmax incurs an additive ε∈[0, β·log(n)] entropy correction relative to its tropical limit. The doubling defect Δ aggregates exactly three such corrections — one in g and two in 2·MFC_β(f) — so the sharp closed-form upper rate predicted by the framework is 3β·log(n). Past cycles falsified the tropical doubling law itself (Δ≠0), so SC5's natural next step is to test whether Δ obeys the *predicted* β·log(n) entropy ceiling.

Taxonomy category: `tropical_discrepancy_finite_beta` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2d88fb78c156ecf4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across all 9 n-values × 30 seeds (270 instances) at β=5, every instance must satisfy Δ(f,β,n) ≤ 3·β·log(n). Support requires max_{n,seed} Δ/B(n) ≤ 1.0 on all 270 instances AND bootstrap 95% CI for the log-log scaling exponent of mean Δ vs n contains 1.0.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.97	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.97	SAFE	SAFE
ALGEBRIZATION	UNCERTAIN	0.95	SAFE	HITS
KARP_LIPTON	SAFE	0.99	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Maslov dequantization tropical Fourier softmax convolution - log-sum-exp tropical polynomial doubling inequality entropy bound - tropical harmonic analysis beta-softmax characters Z_n

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=2.7s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    beta = 5
    n_values = [8, 12, 16, 20, 24, 28, 32, 36, 40]
    results = []

    for n in n_values:
        instances_tested = 0
        max_delta_over_bound = 0

        for _ in range(30):
            f = [random.randint(0, 9) for _ in range(n)]
            g = [beta * math.log(sum(math.exp((f[y] + f[(x - y) % n]) / beta) for y in range(n)) /

            def TFT_beta(h, k):
                exp_sum = 0
                for x in range(n):
                    exp_sum += math.exp(h[x] / beta) * math.exp(-2 * math.pi * k * x / n)
                return beta * math.log(exp_sum)

            MFC_f = min(TFT_beta(f, k) for k in range(1, n))
            MFC_g = min(TFT_beta(g, k) for k in range(1, n))
            delta = abs(MFC_g - 2 * MFC_f)
            bound = 3 * beta * math.log(n)

            instances_tested += 1
            max_delta_over_bound = max(max_delta_over_bound, delta / bound)
```

```
        results.append({
            "n": n,
            "max_delta_over_bound": max_delta_over_bound,
            "instances_tested": instances_tested,
            "conjecture_holds": max_delta_over_bound <= 1.0
        })

    mean_delta_over_bound = sum(result["max_delta_over_bound"] for result in results) / len(result
    std_delta_over_bound = math.sqrt(sum((result["max_delta_over_bound"] - mean_delta_over_bound)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    return {
        "metric_name": "Max Delta Over Bound",
        "metric_value": mean_delta_over_bound,
        "instances_tested": sum(result["instances_tested"] for result in results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": "" if all(result["conjecture_holds"] for result in results) else f"max_d
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
Over Bound', 'metric_value': 0.2687352166673282, 'instances_tested': 270, 'n_max': 40, 'conjecture_ho
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.24593920486738285, 'instances_teste
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.25529857646240417, 'instances_teste
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.2579727947421612, 'instances_tested
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.2621384607564846, 'instances_tested
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.2537786447714733, 'instances_tested
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.2511935744185128, 'instances_tested
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.25949311095586824, 'instances_teste
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.25987547236255815, 'instances_teste
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.25462942709681635, 'instances_teste
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.2538796109332433, 'instances_tested
TRIAL: {'metric_name': 'Max Delta Over Bound', 'metric_value': 0.26432904958395725, 'instances_teste
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test mis-implements two of the three named primitives. TFT_β is defined as β·log(|Σ exp(h(x)/β)·exp(−2πi·kx/n)|) — a complex Fourier sum with absolute value — but the code computes `math.exp(-2*math.pi*k*x/n)` with no imaginary unit and no abs(), yielding a

real exponentially-weighted one-sided sum that is NOT a tropical/softmax Fourier transform. Additionally, g is defined as β·log(Σ exp((f(y)+f(x−y))/β)), but the code divides by n inside the log, subtracting β·log(n) — exactly the kind of

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The critic identifies substantive implementation errors: the code omits the imaginary unit and absolute value in TFT_β (computing a real one-sided exponential sum rather than a Fourier magnitude) and divides by n inside the softmax for g (absorbing the very β·log(n) entropy term the bound is meant to control). The numerical 'support' therefore does not test the stated conjecture. | next: Reimplement TFT_β using complex exponentials with np.abs() and remove the 1/n normalization inside g, then re

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	69112
2	preregistration	claude_max	opus	0	0	4674
3	novelty	claude_max	opus	0	0	3880
4	novelty	claude_max	opus	0	0	6489
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16267
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17212
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10666
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9469
9	critic	claude_max	opus	0	0	24510
10	judge	claude_max	opus	0	0	5111

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 167390 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/04c00d1d0cca.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/04c00d1d0cca.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/04c00d1d0cca.tar.gz` (if generated)